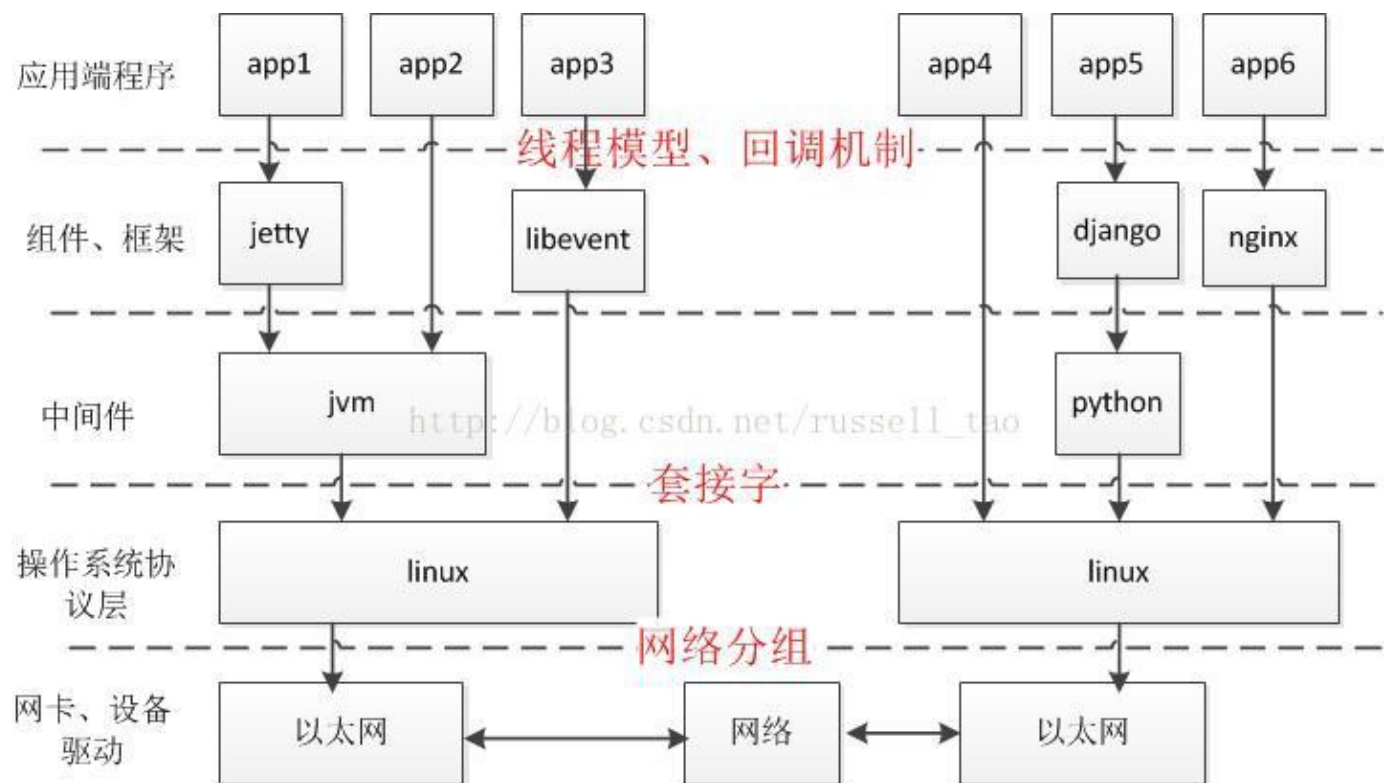


高并发服务器（一）

高并发服务器开发



多路I/O转接服务器

select

1、select函数

1、**select**函数，能监听的文件描述符个数受限于FD_SETSIZE,一般为1024。单纯改变进程打开的文件描述符个数并不能改变**select**监听文件个数。

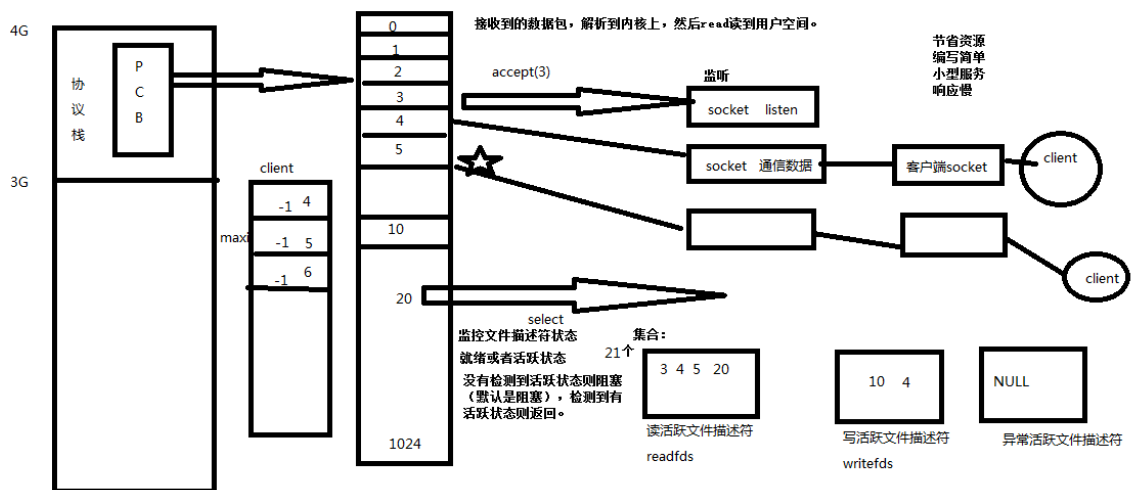
2、解决1024以下客户端时使用**select**是很合适的，**select**采用的是轮询模型，会降低服务器的响应效率。只能用于小型服务器。

```
1 /* According to earlier standards */
2 #include <sys/select.h>
3 #include <sys/time.h>
4 #include <sys/types.h>
5 #include <unistd.h>
6
7
8 int select(int nfds, fd set *readfds, fd set *writefds, fd set *exceptfds, struct time
```

```

9
10
11 nfds: 监控的文件描述符集里最大文件描述符加1, 因此此参数会告诉内核检测前多少个文件描述符的状态
12 readfds: 监控有读数据到达文件描述符集合, 传入传出参数
13 writefds: 监控写数据到达文件描述符集合, 传入传出参数
14 exceptfds: 监控异常发生达文件描述符集合, 如带外数据到达异常, 传入传出参数
15 timeout: 定时阻塞监控时间, 3种情况
16     1.NULL, 永远等下去
17     2.设置timeval, 等待固定时间
18     3.设置timeval里时间均为0, 检查描述字后立即返回, 轮询
19
20
21 struct timeval
22 {
23     long tv_sec; /* seconds */
24     long tv_usec; /* microseconds */
25 };
26
27
28 select集合:
29     void FD_CLR(int fd, fd_set *set); //把某个文件描述符集合里fd置0
30     int FD_ISSET(int fd, fd_set *set); //测试文件描述符集合里fd是否置1
31     void FD_SET(int fd, fd_set *set); //把文件描述符集合里fd位置1
32     void FD_ZERO(fd_set *set); //把文件描述符集合里所有位清0
33
34
35 返回值: 返回就绪文件描述符的个数, 出错返回-1

```



2、select服务端

2.1、server.c

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <sys/types.h>
6 #include <sys/socket.h>
7 #include <sys/select.h>
8 #include <arpa/inet.h>
9 #include <ctype.h>
10 #include <signal.h>
11 #include "wrap.h"
12
13
14 #define SERV_PORT 8090
15 #define MAXLINE 4096
16
17
18
19
20 int main(void)
21 {
22     struct sockaddr_in serv_addr, clie_addr;
23     int sfd, cfd, maxfd, res, len, i, j;
24     fd_set rset, rbset;
25     socklen_t clie_len;
26     char buf[4096];
27
28
29     sfd = Socket(AF_INET, SOCK_STREAM, 0);
30
31
32     memset(&serv_addr, 0, sizeof(serv_addr));
33     serv_addr.sin_family = AF_INET;
34     serv_addr.sin_port = htons(SERV_PORT);
35     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
36
37
```

```

37
38     Bind(sfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
39     Listen(sfd, 128);
40
41
42     FD_ZERO(&rbset); //清零操作
43     FD_SET(sfd, &rbset); //把用于接收客户端链接的sfd, 设置到select中读监控集合
44     maxfd = sfd; //最大文件描述符
45     printf("Accept connect ...\n");
46     while(1)
47     {
48         rset = rbset;
49         res = select(maxfd+1, &rset, NULL, NULL, NULL); //返回活跃文件描述符
50         if(res <= 0)
51             continue;
52         if(FD_ISSET(sfd, &rset)) //监听到有客户端链接
53         {
54             clie_len = sizeof(clie_addr);
55             cfd = Accept(sfd, (struct sockaddr *)&clie_addr, &clie_len);
56             FD_SET(cfd, &rbset); //设置cfd监听
57             maxfd = cfd; //扩大监听文件描述符范围
58             res--;
59         }
60         //处理链接, 轮询
61         for(i = 0; (i <= maxfd) && (res > 0); i++)
62         {
63             if(FD_ISSET(i, &rset))
64             {
65                 len = Read(i, buf, sizeof(buf));
66
67
68                 for(j = 0; j < len; j++)
69                     buf[j] = toupper(buf[j]);
70
71
72                 Write(i, buf, len);
73                 res--;
74             }
75         }
76     }
77
78
79     return 0;
80 }

```

2.2、数组式select案例

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <netinet/in.h>
5 #include <arpa/inet.h>
6 #include "wrap.h"
7
8
9 #define MAXLINE 4096
10 #define SERV_PORT 8090
11
12
13 int main(int argc, char *argv[])
14 {
15     int i, maxi, maxfd, listenfd, connfd, sockfd;
16     int nready, client[FD_SETSIZE]; // FD_SETSIZE 默认为1024
17     ssize_t n;
18     fd_set rset, allset;
19     char buf[MAXLINE];
20     char str[INET_ADDRSTRLEN]; // #define INET_ADDRSTRLEN 16
21     socklen_t clie_len;
22     struct sockaddr_in serv_addr, clie_addr;
23
24
25     listenfd = Socket(AF_INET, SOCK_STREAM, 0);
26
27
28     bzero(&serv_addr, sizeof(serv_addr));
29     serv_addr.sin_family = AF_INET;
30     serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
31     serv_addr.sin_port = htons(SERV_PORT);
32
33
34     Bind(listenfd, (struct sockaddr *)&serv_addr, sizeof(serv_addr));
35
36
```

```

36
37 Listen(listenfd,20);//默认最大128
38
39
40 maxfd = listenfd;//初始化最大值
41 maxi = -1;//client[]的下标
42
43
44 for(i = 0; i < FD_SETSIZE; i++)
45     client[i] = -1;//用-1初始化client[]
46
47
48 FD_ZERO(&allset);
49 FD_SET(listenfd,&allset);//构造select监控文件描述符集
50
51
52 for(;;)
53 {
54     rset = allset;//每次循环都从新设置select监控信号集
55     nready = select(maxfd+1,&rset,NULL,NULL,NULL);
56     if(nready < 0)
57         perr_exit("select err");
58
59
60     //监控是否有新链接
61     if(FD_ISSET(listenfd,&rset))
62     {
63         clie_len = sizeof(clie_addr);
64         connfd = Accept(listenfd,(struct sockaddr *)&clie_addr,&clie_len);
65         printf("received from %s at PORT %d\n",inet_ntop(AF_INET,&clie_addr.sin_
66
67
68         for(i = 0; i < FD_SETSIZE; i++)
69             if(client[i] < 0)
70             {
71                 client[i] = connfd;//保存accept返回的文件描述符到client[]里
72                 break;
73             }
74         //达到select能监控的文件个数上限1024
75         if(i == FD_SETSIZE)
76         {
77             fputs("too many clients\n",stderr);
78             exit(1);
79         }

```

```
80
81
82     FD_SET(connfd,&allset);//添加一个新的文件描述符到监控信号集里
83     if(connfd > maxfd)
84         maxfd = connfd;//select第一个参数需要
85     if(i > maxi)
86         maxi = i;//更新client[]最大下标值
87
88
89     if(--nready == 0)
90         continue;//如果没有更多的就绪文件描述符继续回到上面select阻塞监听，负责处
91
92
93 }
94 for(i = 0; i<= maxi; i++)//检测那个clients有数据就绪
95 {
96     if((sockfd = client[i]) < 0)
97         continue;
98     if(FD_ISSET(sockfd,&rset))
99     {
100         if((n = Read(sockfd,buf,MAXLINE)) == 0)
101         {
102             //client关闭链接时，服务器端也关闭对应链接
103             Close(sockfd);
104             FD_CLR(sockfd,&allset);//解除select监控此文件描述符
105             client[i] = -1;
106         }
107         else
108         {
109             int j;
110             for(j = 0;j < n;j++)
111                 buf[j] = toupper(buf[j]);
112             Write(sockfd,buf,n);
113         }
114         if(--nready == 0)
115             break;
116     }
117 }
118 }
119 Close(listenfd);
120 return 0;
121 }
```

3、pselect函数

```
1 #include <sys/select.h>
2
3
4 int pselect(int nfd, fd_set *readfds, fd_set *writefds, fd_set *exceptfds, const struct timespec
5
6
7 struct timespec
8 {
9     long tv_sec; /* seconds */
10    long tv_nsec; /* nanoseconds */
11 };
12
13
14 用sigmask替代当前进程的阻塞信号集，调用返回后还原原有阻塞信号集
```

poll

1、poll函数

1.1、poll函数（突破1024，修改最大文件描述符）

```
1 #include <poll.h>
2
3
4 int poll(struct pollfd *fds, nfds_t nfd, int timeout);
5
6
7 struct pollfd
8 {
9     int fd; /* 文件描述符*/
10    short events; /* 监控的事件*/
11    short revents; /* 监控事件中满足条件返回的事件*/
12 };
13
14
```



```

15 events:
16 POLLIN 普通或带外优先数据可读,即POLLRDNORM | POLLRDBAND
17 POLLRDNORM-数据可读
18 POLLRDBAND-优先级带数据可读
19 POLLPRI 高优先级可读数据
20
21
22 POLLOUT 普通或带外数据可写
23 POLLWRNORM-数据可写
24 POLLWRBAND-优先级带数据可写
25
26
27 POLLERR 发生错误
28 POLLHUP 发生挂起
29 POLLNVAL 描述字不是一个打开的文件
30
31
32 nfds 监控数组中有多少文件描述符需要被监控
33
34
35 timeout 毫秒级等待
36 -1: 阻塞等待, #define INFTIM -1 Linux中没有定义此宏
37 0: 立即返回, 不阻塞进程
38 >0: 等待指定毫秒数, 如当前系统时间精度不够毫秒, 向上取整
39
40
41 如果poll的结构体中的fd设置为-1时, poll不再监控此pollfd,下次返回时,把revents设置为0。

```

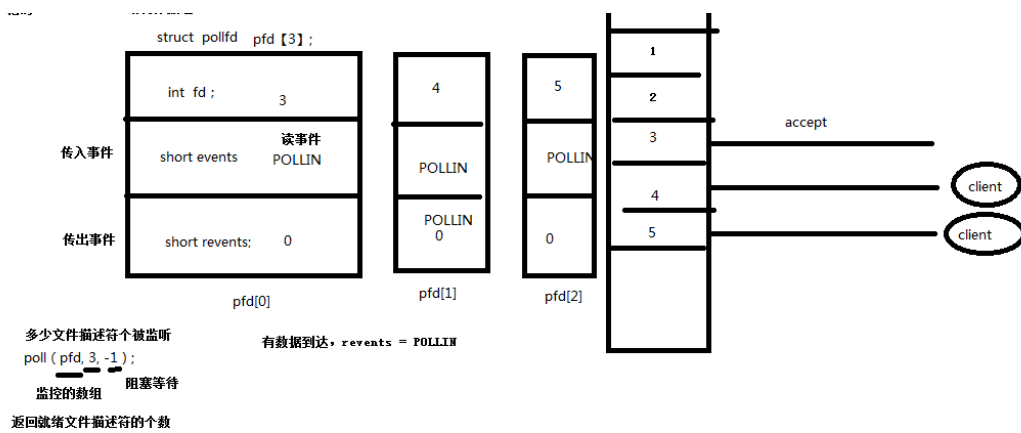
1.2、ppoll函数

ppoll GNU定义了ppoll(非POSIX标准), 可以支持设置信号屏蔽字, 大家可参考poll模型自行实现C/S

```

1 #define _GNU_SOURCE /* See feature_test_macros(7) */
2 #include <poll.h>
3
4
5 int ppoll(struct pollfd *fds, nfds_t nfds, const struct timespec *timeout_ts, const si

```



1.3、poll服务器端

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <string.h>
5 #include <arpa/inet.h>
6 #include <netinet/in.h>
7 #include <poll.h>
8 #include <errno.h>
9 #include "wrap.h"
10
11
12 #define SERV_PORT 8090
13 #define MAXLINE 4096
14 #define OPEN_MAX 1024//打开最大文件描述符
15
16
17
18
19 int main(void)
20 {
21     int i,j,maxi,listenfd,connfd,sockfd;
22     int nready;
23     ssize_t n;
24     char buf[MAXLINE],str[INET_ADDRSTRLEN];
25     socklen_t clilen;
26     struct pollfd client[OPEN_MAX];
27     struct sockaddr_in cliaddr,servaddr;
28

```

```

29
30 listenfd = Socket(AF_INET,SOCK_STREAM,0);
31
32
33 bzero(&servaddr,sizeof(servaddr));
34 servaddr.sin_family = AF_INET;
35 servaddr.sin_addr.s_addr = htonl(INADDR_ANY);
36 servaddr.sin_port = htons(SERV_PORT);
37
38
39 Bind(listenfd,(struct sockaddr *)&servaddr,sizeof(servaddr));
40
41
42 Listen(listenfd,20);
43
44
45 client[0].fd = listenfd;//设置监听文件描述符
46 client[0].events = POLLRDNORM;//listenfd监听普通读事件
47
48
49 for(i = 1;i < OPEN_MAX;i++)
50     client[i].fd = -1;
51 //用-1初始化client[]里剩下元素
52 maxi = 0;
53 //client[]数组有效元素中最大元素下标
54 for(;;)
55 {
56     nready = poll(client,maxi+1,-1);//阻塞，监听设置的数组，传出传入
57     //阻塞，有客户端链接请求
58     if(client[0].revents & POLLRDNORM)//是否有监听到达
59     {
60         clilen = sizeof(cliaddr);
61         connfd = Accept(listenfd,(struct sockaddr *)&cliaddr,&clilen);
62         printf("received from %s at PORT %d\n",inet_ntop(AF_INET,&cliaddr.sin_ad
63         //添加监听文件描述符
64         for(i = 1;i < OPEN_MAX;i++)//设置监听文件描述符
65             if(client[i].fd < 0)
66             {
67                 client[i].fd = connfd;
68                 break;//找到client[]中空闲的位置，存放accept返回的connfd
69
70
71             }
72         if(i == OPEN_MAX)

```

```

72         if(i == OPEN_MAX)
73             perr_exit("too many clients");
74         client[i].events = POLLRDNORM;//设置刚刚返回的connfd,监控读事件
75         if(i > maxi)
76             maxi = i;//更新client[]中最大元素下标
77         if(--nready <= 0)
78             continue;//没有更多就绪事件时,继续回到poll阻塞
79     }
80     for(i = 1;i <= maxi; i++)
81     {
82         //检测client[]
83         if((sockfd = client[i].fd) < 0)
84             continue;
85         if(client[i].revents & (POLLRDNORM | POLLERR))
86         {
87             if((n = Read(sockfd,buf,MAXLINE)) < 0)//读失败
88             {
89                 if(errno == ECONNRESET)
90                 {
91                     //对方丢包了,当收到RST 标志时
92                     //connection reset by client
93                     printf("client[%d] aborted connection\n",i);
94                     Close(sockfd);
95                     client[i].fd = -1;
96                 }
97                 else
98                 {
99                     perr_exit("read error");
100                 }
101             }
102             else if(n == 0)
103             {
104                 //connection closed by client
105                 printf("client[%d] closed connection\n",i);
106                 Close(sockfd);
107                 client[i].fd = -1;
108             }
109             else
110             {
111                 for(j = 0; j < n; j++)
112                     buf[j] = toupper(buf[j]);
113                 Writen(sockfd,buf,n);
114             }
115             if(--nready <= 0)

```

```

116         break;
117     }
118 }
119 }
120 return 0;
121 }

```

压力测试webbench-1.5

./webbench -c 2000 -t 60 <http://192.168.152.128:8090/>

1、socket.c

```

1  /* $Id: socket.c 1.1 1995/01/01 07:11:14 cthuang Exp $
2  *
3  * This module has been modified by Radim Kolar for OS/2 emx
4  */
5
6
7  /*****
8   module:      socket.c
9   program:      popclient
10  SCCS ID:      @(#)socket.c    1.5  4/1/94
11  programmer:   Virginia Tech Computing Center
12  compiler:     DEC RISC C compiler (Ultrix 4.1)
13  environment:  DEC Ultrix 4.3
14  description:  UNIX sockets code.
15  *****/
16
17
18 #include <sys/types.h>
19 #include <sys/socket.h>
20 #include <fcntl.h>
21 #include <netinet/in.h>
22 #include <arpa/inet.h>
23 #include <netdb.h>
24 #include <sys/time.h>
25 #include <string.h>
26 #include <unistd.h>
27 #include <stdio.h>
28 #include <stdlib.h>

```

```

29 #include <stdarg.h>
30
31
32 int Socket(const char *host, int clientPort)
33 {
34     int sock;
35     unsigned long inaddr;
36     struct sockaddr_in ad;
37     struct hostent *hp;
38
39
40     memset(&ad, 0, sizeof(ad));
41     ad.sin_family = AF_INET;
42
43
44     inaddr = inet_addr(host);
45     if (inaddr != INADDR_NONE)
46         memcpy(&ad.sin_addr, &inaddr, sizeof(inaddr));
47     else
48     {
49         hp = gethostbyname(host);
50         if (hp == NULL)
51             return -1;
52         memcpy(&ad.sin_addr, hp->h_addr, hp->h_length);
53     }
54     ad.sin_port = htons(clientPort);
55
56
57     sock = socket(AF_INET, SOCK_STREAM, 0);
58     if (sock < 0)
59         return sock;
60     if (connect(sock, (struct sockaddr *)&ad, sizeof(ad)) < 0)
61         return -1;
62     return sock;
63 }

```

2、webbench.c

```
2  * (C) Radim Kolar 1997-2004
3  * This is free software, see GNU Public License version 2 for
4  * details.
5  *
6  * Simple forking WWW Server benchmark:
7  *
8  * Usage:
9  *   webbench --help
10 *
11 * Return codes:
12 *   0 - success
13 *   1 - benchmark failed (server is not on-line)
14 *   2 - bad param
15 *   3 - internal error, fork failed
16 *
17 */
18 #include "socket.c"
19 #include <unistd.h>
20 #include <sys/param.h>
21 #include <rpc/types.h>
22 #include <getopt.h>
23 #include <strings.h>
24 #include <time.h>
25 #include <signal.h>
26
27
28 /* values */
29 volatile int timerexpired=0;
30 int speed=0;
31 int failed=0;
32 int bytes=0;
33 /* globals */
34 int http10=1; /* 0 - http/0.9, 1 - http/1.0, 2 - http/1.1 */
35 /* Allow: GET, HEAD, OPTIONS, TRACE */
36 #define METHOD_GET 0
37 #define METHOD_HEAD 1
38 #define METHOD_OPTIONS 2
39 #define METHOD_TRACE 3
40 #define PROGRAM_VERSION "1.5"
41 int method=METHOD_GET;
42 int clients=1;
43 int force=0;
44 int force_reload=0;
```

```

45 int proxyport=80;
46 char *proxyhost=NULL;
47 int benchtime=30;
48 /* internal */
49 int mypipe[2];
50 char host[MAXHOSTNAMELEN];
51 #define REQUEST_SIZE 2048
52 char request[REQUEST_SIZE];
53
54
55 static const struct option long_options[]=
56 {
57     {"force",no_argument,&force,1},
58     {"reload",no_argument,&force_reload,1},
59     {"time",required_argument,NULL,'t'},
60     {"help",no_argument,NULL,'?'},
61     {"http09",no_argument,NULL,'9'},
62     {"http10",no_argument,NULL,'1'},
63     {"http11",no_argument,NULL,'2'},
64     {"get",no_argument,&method,METHOD_GET},
65     {"head",no_argument,&method,METHOD_HEAD},
66     {"options",no_argument,&method,METHOD_OPTIONS},
67     {"trace",no_argument,&method,METHOD_TRACE},
68     {"version",no_argument,NULL,'V'},
69     {"proxy",required_argument,NULL,'p'},
70     {"clients",required_argument,NULL,'c'},
71     {NULL,0,NULL,0}
72 };
73
74
75 /* prototypes */
76 static void benchcore(const char* host,const int port, const char *request);
77 static int bench(void);
78 static void build_request(const char *url);
79
80
81 static void alarm_handler(int signal)
82 {
83     timerexpired=1;
84 }
85
86
87 static void usage(void)
88 {

```



```

89     fprintf(stderr,
90         "webbench [option]... URL\n"
91         "  -f|--force           Don't wait for reply from server.\n"
92         "  -r|--reload          Send reload request - Pragma: no-cache.\n"
93         "  -t|--time <sec>      Run benchmark for <sec> seconds. Default 30.\n"
94         "  -p|--proxy <server:port> Use proxy server for request.\n"
95         "  -c|--clients <n>      Run <n> HTTP clients at once. Default one.\n"
96         "  -9|--http09           Use HTTP/0.9 style requests.\n"
97         "  -1|--http10           Use HTTP/1.0 protocol.\n"
98         "  -2|--http11           Use HTTP/1.1 protocol.\n"
99         "  --get                 Use GET request method.\n"
100        "  --head                Use HEAD request method.\n"
101        "  --options             Use OPTIONS request method.\n"
102        "  --trace               Use TRACE request method.\n"
103        "  -?|-h|--help         This information.\n"
104        "  -V|--version          Display program version.\n"
105        );
106 };
107 int main(int argc, char *argv[])
108 {
109     int opt=0;
110     int options_index=0;
111     char *tmp=NULL;
112
113
114     if(argc==1)
115     {
116         usage();
117         return 2;
118     }
119
120
121     while((opt=getopt_long(argc,argv,"912Vfrt:p:c:?h",long_options,&options_index))!=EOF)
122     {
123         switch(opt)
124         {
125             case 0 : break;
126             case 'f': force=1;break;
127             case 'r': force_reload=1;break;
128             case '9': http10=0;break;
129             case '1': http10=1;break;
130             case '2': http10=2;break;
131             case 'V': printf(PROGRAM_VERSION"\n");exit(0);
132             case 't': benchtime=atoi(optarg);break;

```

```

132     case 't': benchtime=atoi(optarg);break;
133     case 'p':
134         /* proxy server parsing server:port */
135         tmp=strrchr(optarg,':');
136         proxyhost=optarg;
137         if(tmp==NULL)
138         {
139             break;
140         }
141         if(tmp==optarg)
142         {
143             fprintf(stderr,"Error in option --proxy %s: Missing hostname.\n");
144             return 2;
145         }
146         if(tmp==optarg+strlen(optarg)-1)
147         {
148             fprintf(stderr,"Error in option --proxy %s Port number is missing.\n");
149             return 2;
150         }
151         *tmp='\0';
152         proxyport=atoi(tmp+1);break;
153     case ':':
154     case 'h':
155     case '?': usage();return 2;break;
156     case 'c': clients=atoi(optarg);break;
157 }
158 }
159
160
161 if(optind==argc) {
162     fprintf(stderr,"webbench: Missing URL!\n");
163     usage();
164     return 2;
165 }
166
167
168 if(clients==0) clients=1;
169 if(benchtime==0) benchtime=60;
170 /* Copyright */
171 fprintf(stderr,"Webbench - Simple Web Benchmark "PROGRAM_VERSION"\n");
172     "Copyright (c) Radim Kolar 1997-2004, GPL Open Source Software.\n"
173 );
174 build_request(argv[optind]);
175 /* print bench info */

```

```
176 printf("\nBenchmarking: ");
177 switch(method)
178 {
179     case METHOD_GET:
180     default:
181         printf("GET");break;
182     case METHOD_OPTIONS:
183         printf("OPTIONS");break;
184     case METHOD_HEAD:
185         printf("HEAD");break;
186     case METHOD_TRACE:
187         printf("TRACE");break;
188 }
189 printf(" %s",argv[optind]);
190 switch(http10)
191 {
192     case 0: printf(" (using HTTP/0.9)");break;
193     case 2: printf(" (using HTTP/1.1)");break;
194 }
195 printf("\n");
196 if(clients==1) printf("1 client");
197 else
198     printf("%d clients",clients);
199
200
201 printf(", running %d sec", benchtime);
202 if(force) printf(", early socket close");
203 if(proxyhost!=NULL) printf(", via proxy server %s:%d",proxyhost,proxyport);
204 if(force_reload) printf(", forcing reload");
205 printf(".\n");
206 return bench();
207 }
208
209
210 void build_request(const char *url)
211 {
212     char tmp[10];
213     int i;
214
215
216     bzero(host,MAXHOSTNAMELEN);
217     bzero(request,REQUEST_SIZE);
218
219
```

```

220 if(force_reload && proxyhost!=NULL && http10<1) http10=1;
221 if(method==METHOD_HEAD && http10<1) http10=1;
222 if(method==METHOD_OPTIONS && http10<2) http10=2;
223 if(method==METHOD_TRACE && http10<2) http10=2;
224
225
226 switch(method)
227 {
228     default:
229     case METHOD_GET: strcpy(request,"GET");break;
230     case METHOD_HEAD: strcpy(request,"HEAD");break;
231     case METHOD_OPTIONS: strcpy(request,"OPTIONS");break;
232     case METHOD_TRACE: strcpy(request,"TRACE");break;
233 }
234
235
236 strcat(request, " ");
237
238
239 if(NULL==strstr(url, "://"))
240 {
241     fprintf(stderr, "\n%s: is not a valid URL.\n",url);
242     exit(2);
243 }
244 if(strlen(url)>1500)
245 {
246     fprintf(stderr,"URL is too long.\n");
247     exit(2);
248 }
249 if(proxyhost==NULL)
250     if (0!=strncasecmp("http://",url,7))
251     { fprintf(stderr,"\nOnly HTTP protocol is directly supported, set --proxy
252         exit(2);
253     }
254 /* protocol/host delimiter */
255 i=strstr(url, "://")-url+3;
256 /* printf("%d\n",i); */
257
258
259 if(strchr(url+i, '/')==NULL) {
260     fprintf(stderr,"\nInvalid URL syntax - hostname don't
261     exit(2);
262 }

```

```

263 if(proxyhost==NULL)
264 {
265     /* get port from hostname */
266     if(index(url+i,':')!=NULL &&
267         index(url+i,':')<index(url+i,'/'))
268     {
269         strncpy(host,url+i,strchr(url+i,':')-url-i);
270         bzero(tmp,10);
271         strncpy(tmp,index(url+i,':')+1,strchr(url+i,'/')->index(url+i,':')-1);
272         /* printf("tmp=%s\n",tmp); */
273         proxyport=atoi(tmp);
274         if(proxyport==0) proxyport=80;
275     } else
276     {
277         strncpy(host,url+i,strcspn(url+i,"/"));
278     }
279     // printf("Host=%s\n",host);
280     strcat(request+strlen(request),url+i+strcspn(url+i,"/"));
281 } else
282 {
283     // printf("ProxyHost=%s\nProxyPort=%d\n",proxyhost,proxyport);
284     strcat(request,url);
285 }
286 if(http10==1)
287     strcat(request," HTTP/1.0");
288 else if (http10==2)
289     strcat(request," HTTP/1.1");
290 strcat(request,"\r\n");
291 if(http10>0)
292     strcat(request,"User-Agent: WebBench "PROGRAM_VERSION"\r\n");
293 if(proxyhost==NULL && http10>0)
294 {
295     strcat(request,"Host: ");
296     strcat(request,host);
297     strcat(request,"\r\n");
298 }
299 if(force_reload && proxyhost!=NULL)
300 {
301     strcat(request,"Pragma: no-cache\r\n");
302 }
303 if(http10>1)
304     strcat(request,"Connection: close\r\n");
305 /* add empty line at end */
306 if(http10>0) strcat(request,"\r\n");

```

```

307 // printf("Req=%s\n",request);
308 }
309
310
311 /* vraci system rc error kod */
312 static int bench(void)
313 {
314     int i,j,k;
315     pid_t pid=0;
316     FILE *f;
317
318
319     /* check availability of target server */
320     i=Socket(proxyhost==NULL?host:proxyhost,proxyport);
321     if(i<0) {
322         fprintf(stderr,"\nConnect to server failed. Aborting benchmark.\n");
323         return 1;
324     }
325     close(i);
326     /* create pipe */
327     if(pipe(mypipe))
328     {
329         perror("pipe failed.");
330         return 3;
331     }
332
333
334     /* not needed, since we have alarm() in childrens */
335     /* wait 4 next system clock tick */
336     /*
337     cas=time(NULL);
338     while(time(NULL)==cas)
339         sched_yield();
340     */
341
342
343     /* fork childs */
344     for(i=0;i<clients;i++)
345     {
346         pid=fork();
347         if(pid <= (pid_t) 0)
348         {
349             /* child process or error*/

```

```

350         sleep(1); /* make childs faster */
351         break;
352     }
353 }
354
355
356 if( pid< (pid_t) 0)
357 {
358     fprintf(stderr,"problems forking worker no. %d\n",i);
359     perror("fork failed.");
360     return 3;
361 }
362
363
364 if(pid== (pid_t) 0)
365 {
366     /* I am a child */
367     if(proxyhost==NULL)
368         benchcore(host,proxyport,request);
369     else
370         benchcore(proxyhost,proxyport,request);
371
372
373     /* write results to pipe */
374     f=fdopen(mypipe[1],"w");
375     if(f==NULL)
376     {
377         perror("open pipe for writing failed.");
378         return 3;
379     }
380     /* fprintf(stderr,"Child - %d %d\n",speed,failed); */
381     fprintf(f,"%d %d %d\n",speed,failed,bytes);
382     fclose(f);
383     return 0;
384 } else
385 {
386     f=fdopen(mypipe[0],"r");
387     if(f==NULL)
388     {
389         perror("open pipe for reading failed.");
390         return 3;
391     }
392     setvbuf(f,NULL,_IONBF,0);
393     speed=0;

```

```

394         failed=0;
395         bytes=0;
396
397
398         while(1)
399         {
400             pid=fscanf(f,"%d %d %d",&i,&j,&k);
401             if(pid<2)
402             {
403                 fprintf(stderr,"Some of our childrens died.\n");
404                 break;
405             }
406             speed+=i;
407             failed+=j;
408             bytes+=k;
409             /* fprintf(stderr,"*Knock* %d %d read=%d\n",speed,failed,pid); */
410             if(--clients==0) break;
411         }
412         fclose(f);
413
414
415         printf("\nSpeed=%d pages/min, %d bytes/sec.\nRequests: %d susceed, %d failed.\n",
416             (int)((speed+failed)/(benchtime/60.0f)),
417             (int)(bytes/(float)benchtime),
418             speed,
419             failed);
420     }
421     return i;
422 }
423
424
425 void benchcore(const char *host,const int port,const char *req)
426 {
427     int rlen;
428     char buf[1500];
429     int s,i;
430     struct sigaction sa;
431
432
433     /* setup alarm signal handler */
434     sa.sa_handler=alarm_handler;
435     sa.sa_flags=0;
436     if(sigaction(SIGALRM,&sa,NULL))
437         exit(3);

```



```

437     close(s);
438     alarm(benchmark);
439
440
441     rlen=strlen(req);
442     nexttry:while(1)
443     {
444         if(timerexpired)
445         {
446             if(failed>0)
447             {
448                 /* fprintf(stderr,"Correcting failed by signal\n"); */
449                 failed--;
450             }
451             return;
452         }
453         s=Socket(host,port);
454         if(s<0) { failed++;continue;}
455         if(rlen!=write(s,req,rlen)) {failed++;close(s);continue;}
456         if(http10==0)
457             if(shutdown(s,1)) { failed++;close(s);continue;}
458         if(force==0)
459         {
460             /* read all available data from socket */
461             while(1)
462             {
463                 if(timerexpired) break;
464                 i=read(s,buf,1500);
465                 /* fprintf(stderr,"%d\n",i); */
466                 if(i<0)
467                 {
468                     failed++;
469                     close(s);
470                     goto nexttry;
471                 }
472                 else
473                     if(i==0) break;
474                     else
475                         bytes+=i;
476             }
477         }
478         if(close(s)) {failed++;continue;}
479         speed++;
480     }

```

481 }

